Abschlussbericht: DirtNet

Visuelle Schmutz- und Objekterkennung für autonome Reinigungsroboter

Shufang Tao (3824652) · Yuxin Song (3820779) · Yuyang Ren (3762594)

Stand des GitHub-Repositories: Commit **69bb7c9**, 6. Juli 2026

Kurzfassung

Das Projekt untersucht die bodennahe Erkennung von Schmutz und typischen Bürogegenständen auf Basis der YOLO-Familie. Ausgehend von einer YOLOv3-Reproduktion wurde eine systematische Versuchsreihe bis zu YOLO11 durchgeführt. Die stärksten synthetischen Validierungsergebnisse werden auf realen Bildern deutlich unterschritten. Für die binäre Aufgabe wird deshalb YOLO11s als praxisnahes Modell empfohlen; für die Zehnklassenaufgabe bleibt das erste YOLO11m-Modell der belastbarste Ausgangspunkt. Der Bericht fasst Konzept, Implementierung, Experimente, Grenzen und nächste Schritte zusammen.

1 Einleitung und Motivation

Autonome Reinigungsroboter müssen nicht nur Hindernisse erkennen, sondern auch kleine, unregelmäßige und kontrastarme Verschmutzungen lokalisieren. Schmutz besitzt keine stabile Form; Reflexionen und Bodenstrukturen erzeugen ähnliche Bildmuster, und die Kameraperspektive liegt nahe am Boden. Ziel des Projekts war eine reproduzierbare Detektionspipeline für *dirt* und Büroobjekte sowie die Untersuchung, wie Modellgröße, Auflösung, Optimierer und Datendomäne die Generalisierung beeinflussen.

2 Literatur und Grundlagen

Die fachliche Referenz ist *DirtNet: Visual Dirt Detection for Autonomous Cleaning Robots* (ICRA 2020). Methodisch basiert die Implementierung auf Ultralytics YOLOv3 und YOLO11. Bewertet wird mit Precision, Recall, mAP@0.5 und mAP@0.5:0.95. Während mAP@0.5 grobe korrekte Lokalisierungen belohnt, reagiert mAP@0.5:0.95 stärker auf ungenaue Boxen. Für den Roboterbetrieb sind zusätzlich Laufzeit, Fehlalarmrate und der Anteil übersehener Verschmutzungen relevant.

3 Konzept, Methoden und Implementierung

Die Arbeit gliedert sich in eine binäre Aufgabe (*dirt* gegenüber *object*) und eine Zehnklassenaufgabe mit Schmutz sowie neun Büroobjektklassen. Für Multiclass wurden 860 synthetische Trainingsbilder mit 8.626 Instanzen, 296 synthetische Validierungsbilder mit 2.962 Instanzen und 80 ausschließlich zum Testen verwendete reale Bilder mit 644 Instanzen eingesetzt. Die Klassenverteilung ist unausgeglichen: *dirt* dominiert mit 5.161 Trainingsinstanzen, während *scissors* nur 114 Instanzen besitzt.

Split	Bilder	Instanzen	Verwendung
Train (synthetisch)	860	8.626	Parameterlernen
Validation (synthetisch)	296	2.962	Checkpoint-Auswahl
test_real_new	80	644	unabhängiger Realtest

Die Daten wurden in YOLO-Annotationen konvertiert und über YAML-Dateien reproduzierbar eingebunden. Das formale Multiclass-Basismodell ist YOLO11m mit COCO-Vortraining, Eingabegröße 1024, Batchgröße 2, 30 Epochen, AMP und deterministischem Seed 0. Trainiert wurde auf einer RTX 3060 Laptop GPU mit 6 GB. Für die binären Endkandidaten wurden YOLO11s und YOLO11m bei 1280 Pixeln, SGD und 30 Epochen verglichen.

4 Experimente und Auswertung

4.1 Binäre Detektion

Auf der Validierung erreicht YOLO11m 97,4 % Precision, 96,2 % Recall, 98,3 % mAP50 und 92,3 % mAP50-95. Auf dem unabhängigen Realtest kehrt sich die Rangfolge um: YOLO11s erzielt 80,6 % mAP50 und 56,2 % mAP50-95 bei etwa 10 ms Inferenzzeit, YOLO11m nur 77,8 % beziehungsweise 53,2 % bei 21,4 ms. YOLO11m ist präziser, übersieht aber mehr Ziele. Für den praktischen Einsatz wird daher YOLO11s empfohlen.

Modell / Split	P	R	mAP50	mAP50-95	ms/Bild
YOLO11s / Realtest	0,781	0,772	0,806	0,562	10,0
YOLO11m / Realtest	0,835	0,704	0,778	0,532	21,4

4.2 Multiclass-Basismodell

Das YOLO11m-Basismodell benötigte rund 84,1 Minuten. Der beste Checkpoint entstand in Epoche 29 und erreicht auf der synthetischen Validierung 0,9535 Precision, 0,9305 Recall, 0,9608 mAP50 und 0,8136 mAP50-95. Auf realen Bildern fallen die Werte deutlich. Der mAP50-Rückgang beträgt 0,224, bei mAP50-95 sogar 0,260.

Split	Precision	Recall	mAP50	mAP50-95
Synthetische Validierung	0,953	0,930	0,961	0,814
Realer Test	0,665	0,776	0,737	0,554
Absolute Abnahme	0,288	0,154	0,224	0,260

Besonders robust sind *pens* (mAP50 0,986) und *scissors* (0,922). Schwach sind *other* (0,404), *rulers* (0,490) und *usb_sticks* (0,520). Für *dirt* werden 0,852 Precision, 0,706 Recall und 0,773 mAP50 gemessen; ungefähr 29 % der realen Schmutzinstanzen bleiben unerkannt.

4.3 Fine-Tuning, Modellgröße und Laufzeit

Ein zusätzliches AdamW-Fine-Tuning über 15 Epochen steigerte alle synthetischen Kennzahlen, verschlechterte auf dem Realtest aber Recall (0,776 auf 0,721) und mAP50 (0,737 auf 0,726). Das bestätigt, dass weitere Optimierung auf derselben synthetischen Verteilung das Domänenproblem nicht löst.

Multiclass-Modell	P	R	mAP50	mAP50-95
Baseline, Realtest	0,665	0,776	0,737	0,554
Fine-Tuning, Realtest	0,726	0,721	0,726	0,557

Bei YOLOv8 steigt der Validierungs-mAP50-95 mit der Modellgröße von 0,710 (Nano) über 0,796 (Small) auf 0,813 (Medium); zugleich steigt die Trainingszeit von 11,6 auf 169,7 Minuten. Mehr Kapazität hilft innerhalb der Validierungsdomäne, ist aber kein Ersatz für reale Trainingsdaten.

YOLOv8	mAP50-95	Training
Nano	0,710	11,6 min
Small	0,796	78,6 min
Medium	0,813	169,7 min

Das Multiclass-YOLO11m benötigt insgesamt etwa 33,2 ms pro Bild und erreicht damit ungefähr 30 FPS. Eine ruhige Kameraführung und kurze Stopps in Objektnähe verbesserten im Video-Demo die Stabilität; starke Drehungen erzeugten Bewegungsunschärfe.

4.4 Einordnung und Grenzen

Die zentrale Fehlerquelle ist der Domänenunterschied: reale Bilder enthalten schlechte Beleuchtung, Reflexionen, Sensorrauschen, Kratzer und Materialvarianten, die in den synthetischen Daten nicht ausreichend vorkommen. Zudem enthält der Realtest keine leeren Bodenbilder. Die Fehlalarmrate im kontinuierlichen Betrieb kann daher nicht belastbar angegeben werden. Die 80 Testbilder dürfen nicht für Training oder Hyperparameterwahl wiederverwendet werden, solange kein neuer unabhängiger Realtest vorliegt.

5 Zusammenfassung und Ausblick

Das Projekt liefert eine reproduzierbare YOLO-Pipeline, umfangreiche Modellvergleiche und einen echtbildbasierten Generalisierungstest. Für die binäre Praxisanwendung ist YOLO11s der beste Kompromiss aus Recall, mAP und Geschwindigkeit. Für Multiclass bleibt das erste YOLO11m-Modell der empfohlene Baseline-Checkpoint; das Fine-Tuning wurde wegen schlechterer Realtest-Balance verworfen.

Die wichtigste nächste Maßnahme ist nicht ein größeres Modell, sondern ein getrenntes reales Trainings- und Validierungsset. Ergänzend sollten schwierige Negativbilder, mehr Beispiele schwacher Klassen, Beleuchtungs- und Rauschaugmentation, eine klarere Definition der Klasse *other* sowie schwellenwertabhängige Tests auf sauberen Böden aufgenommen werden.

Literatur und Projektquellen

[1] Bormann et al.: *DirtNet: Visual Dirt Detection for Autonomous Cleaning Robots*, ICRA 2020.
[2] Ultralytics YOLOv3 Community Implementation und Ultralytics YOLO11.
[3] Projekt-Repository: https://github.com/ipa-robotics-courses/2026_p02_dirt_detection, Stand Commit 69bb7c9.
[4] Interne Ergebnisdokumente: yolo11_final_experiment_summary.md, yolo11_multiclass_training_summary.md, yolo11_multiclass_real_test_analysis.md und yolo11_multiclass_finetuning_analysis.md.